More practice - Two pointers and sliding window

4 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

2	2
MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

More practice - Two pointers and sliding window | 4 questions | tap a title to jump to it

M	1. Run-Length Vector Storage + Dot Product	1Point3Acres	2026-01-21
M	2. Sum of Good Subarrays (Adjacent Diff ±1)	1Point3Acres	2026-02-08
H	3. Necklace Cut into Two Halves with Equal D/R Counts	1Point3Acres	2026-05-03
H	4. Find longest duplicated substring	PracHub	2025-12-08

STUDY PATH: question 118 of 167 (more practice)

Priority score 1.7 | Two pointers and sliding window | asked 272 days ago | frequency Single report

MEDIUM | 1Point3Acres | phone-screen, onsite-coding | 1 of 4

Run-Length Vector Storage + Dot Product

ASKED 2026-01-21 | 272 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 5 | 30 DAYS 0 | PRIORITY 173/228

ABOUT THIS QUESTION

Topic: More practice - Two pointers and sliding window

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1570 Dot Product of Two Sparse Vectors

Frequency: Single report | Last asked: 2026-01-21 | Roles: SWE, INFRA-ENG

Original: https://www.1point3acres.com/interview/problems/company/google/compressed-vector-dot-product

QUESTION

EXAMPLES

```
Logical A = [3, 3, 3, 0, 0, 0, 4, 4]
Logical B = [1, 1, 1, 1, 2, 2, 2, 2]
Encoded A (RLE): [(3,3),(0,3),(4,2)]
Encoded B (RLE): [(1,4),(2,4)]
Dot product = 3·1·3 + 3·1·0 + 0·1·0 + 0·2·0 + 4·2·2 =
    9 + 0 + 16 = 25
```

APPROACH

Memorize the two-pointer RLE walk pattern (it generalizes to interval intersection, range addition, etc.).
Practice articulating when RLE wins vs sparse (O(distinct_runs) vs O(nonzeros)).
Have a one-liner for sparse case: store (idx -> value) map; iterate over the smaller map and look up the other.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the encoding format (value, count pairs), whether runs of zeros are stored, the logical length of both vectors (must be equal), and whether updates are needed.

BRUTE FORCE TO SAY OUT LOUD

Expand both vectors and multiply element by element. O(n) memory.

OPTIMAL IDEA

Two pointers over the run lists. Take the overlap, which is min of the two remaining counts, add value_a * value_b * overlap, subtract the overlap from both, and advance whichever run is exhausted. O(r1 + r2), O(1) extra space.

DRY-RUN THESE CASES

The example giving 25. One vector of a single run. Runs that end at the same time. All zeros.

INTERVIEW-DAY PLAN

LIKELY FOLLOW-UPS

A sparse vector against an RLE vector, setting an element, or dot product of k vectors.

PITFALLS

Advancing only one pointer when both runs finish together. Overflowing the product for large counts.

HOW TO THINK ABOUT IT

A run is a stretch where both numbers are constant, so the dot product of two runs is value times value times the length of their overlap. Walk the two run lists like the merge step of merge sort: consume the smaller remaining length from both, and advance whichever run you finished.

Basis: LeetCode 1570, Dot Product of Two Sparse Vectors, with run-length encoding as the compression.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def decode(runs):
    dense = []
    for value, count in runs:
        dense += [value] * count
    return dense
def brute(runs_a, runs_b):
    a, b = decode(runs_a), decode(runs_b)
    return sum(x * y for x, y in zip(a, b))
```

TIME	SPACE
O(logical length)	O(logical length)

WHY IT IS SLOW

It needs memory and time proportional to the logical length, which can be 10^9. Walking the two run lists together with two pointers costs only the number of runs.

OPTIMAL SOLUTION · Two-pointer over runs

TIME	SPACE
O(r1 + r2)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if one vector is mostly zeros and stored sparsely, and the other as runs?

Iterate the sparse side's nonzero indices and binary-search the run list for each index to read the other value: O(s log r). Choose the smaller representation to drive the loop.

FOLLOW-UP 2 How should the data be updated if you set element i?

Locate the run containing i (binary search over prefix run lengths), then split it into at most three runs and merge equal neighbours. That is O(log r) to find plus O(r) in the worst case to splice a list; use a balanced tree of runs for frequent updates.

FOLLOW-UP 3 One vector is a sparse map (index to value) and the other is run-length encoded.

Iterate the smaller structure (the sparse non-zeros) and binary search the run list for each index using prefix run lengths: O(s log r) instead of expanding anything.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Run-Length Vector Storage + Dot Product
#
# Examples: Logical A = [3, 3, 3, 0, 0, 0, 4, 4] Logical B
# = [1, 1, 1, 1, 2, 2, 2, 2] Encoded A (RLE):
# [(3,3),(0,3),(4,2)] Encoded B (RLE): [(1,4),(2,4)] Dot
# product = 3·1·3 + 3·1·0 + 0·1·0 + 0·2·0 + 4·2·2 = 9 + 0
# + 16 = 25
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if one vector is mostly zeros and stored
# sparsely, and the other as runs?
# 2. How should the data be updated if you set element i?
# 3. One vector is a sparse map (index to value) and the
# other is run-length encoded.
#
# INTERVIEW NOTES
# A run is a stretch where both numbers are constant, so
# the dot product of two runs is value times value times
# the length of their overlap. Walk the two run lists like
# the merge step of merge sort: consume the smaller
# remaining length from both, and advance whichever run
# you finished.
#
# BRUTE FORCE (say it first): Expand both vectors and
# multiply
# Decompress each run into repeated values and add up the
# element-wise products. Correct and the oracle; with
# logical lengths like 10^9 it cannot even be stored. Time
# O(logical length). Space O(logical length).


from bisect import bisect_right
from itertools import accumulate


# BRUTE FORCE (say it first): Expand both vectors and
# multiply | time O(logical length)
def rle_dot_expand(a, b):
    x = [v for v, c in a for _ in range(c)]
    y = [v for v, c in b for _ in range(c)]
    return sum(p * q for p, q in zip(x, y))


# BASE QUESTION (optimal): Two-pointer over runs
# Idea: Each run is (value, count). Compare the remaining
# lengths of the two current runs, multiply the values by
# the overlap, then advance whichever run was exhausted
# (or both).
def rle_dot(a, b):
    # two pointers over the runs: never expand anything
    i = j = 0
    left_a = a[0][1] if a else 0
    left_b = b[0][1] if b else 0
    total = 0
    while i < len(a) and j < len(b):
        # both runs overlap for min(left_a, left_b)
        # positions
        take = min(left_a, left_b)
        total += a[i][0] * b[j][0] * take
        left_a -= take
        left_b -= take
        if left_a == 0:
            i += 1
            left_a = a[i][1] if i < len(a) else 0
        if left_b == 0:
            j += 1
            left_b = b[j][1] if j < len(b) else 0
    return total


# FOLLOW-UP 1: What if one vector is mostly zeros and
# stored sparsely, and the other as runs?
# Answer: Iterate the sparse side's nonzero indices and
# binary-search the run list for each index to read the
# other value: O(s log r). Choose the smaller
# representation to drive the loop.
# FOLLOW-UP 3: One vector is a sparse map (index to value)
# and the other is run-length encoded.
# Answer: Iterate the smaller structure (the sparse non-
# zeros) and binary search the run list for each index
# using prefix run lengths: O(s log r) instead of
# expanding anything.
def sparse_dot_rle(sparse, runs):
    # sparse: {index: value}; runs: RLE of the other
    # vector
    ends = list(accumulate(count for _, count in runs))
    total = 0
    # iterate the small side; binary search the runs
    for index, value in sparse.items():
        k = bisect_right(ends, index)
        if k < len(runs):
            total += value * runs[k][0]
    return total


# FOLLOW-UP 2: How should the data be updated if you set
# element i?
# Answer: Locate the run containing i (binary search over
# prefix run lengths), then split it into at most three
# runs and merge equal neighbours.
def set_at(runs, index, value):
    # update one position: split a run, then merge
    # neighbours
    flat, pos = [], 0
    for v, count in runs:
        if pos <= index < pos + count:
            flat += [(v, index - pos), (value, 1),
                     (v, pos + count - index - 1)]
        else:
            flat.append((v, count))
        pos += count
    merged = []
    for v, count in flat:
        if count == 0:
            continue
        if merged and merged[-1][0] == v:
            merged[-1] = (v, merged[-1][1] + count)
        else:
```

```
            merged.append((v, count))
    return merged


# HELPERS for the checks below
def rle_encode(values):
    runs = []
    for v in values:
        if runs and runs[-1][0] == v:
            runs[-1][1] += 1
        else:
            runs.append([v, 1])
    return [tuple(r) for r in runs]


def rle_decode(runs):
    return [v for v, count in runs for _ in range(count)]
```

```
if __name__ == "__main__":
    A = [3, 3, 3, 0, 0, 0, 4, 4]
    B = [1, 1, 1, 1, 2, 2, 2, 2]
    ra, rb = rle_encode(A), rle_encode(B)
    assert ra == [(3, 3), (0, 3), (4, 2)]
    assert rle_decode(ra) == A
    # 3*1*3 + 0 + 4*2*2 = 25
    assert rle_dot(ra, rb) == 25
    assert rle_dot([], rb) == 0
    dense = sum(x * y for x, y in zip(A, B))
    assert dense == 25
    # sparse vector with non-zeros at 0 and 7
    sparse = {0: 2, 7: 5}
    assert sparse_dot_rle(sparse, rb) == 2 * 1 + 5 * 2
    # setting a value splits and merges runs
    assert set_at(ra, 4, 3) == [(3, 3), (0, 1), (3, 1),
                                (0, 1), (4, 2)]
    assert set_at([(1, 3)], 1, 1) == [(1, 3)]
```

NOTES FROM CANDIDATES

Two-pointer walk over the two RLE encodings: at each step compute the overlap length min(remainA, remainB), multiply values, advance the pointer whose run is exhausted.
Mention compression-friendly variations: sparse format wins when most entries are 0 (just enumerate non-zero indices and skip zeros entirely in the dot product).
Interviewer cared more about clean data-structure choice and the walk over both encodings than about absolute optimality.

PREPARATION

Memorize the two-pointer RLE walk pattern (it generalizes to interval intersection, range addition, etc.).
Practice articulating when RLE wins vs sparse (O(distinct_runs) vs O(nonzeros)).
Have a one-liner for sparse case: store (idx -> value) map; iterate over the smaller map and look up the other.

STUDY PATH: question 119 of 167 (more practice)

Priority score 1.5 | Two pointers and sliding window | asked 254 days ago | frequency Single report

MEDIUM | 1Point3Acres | onsite-coding | 2 of 4

Sum of Good Subarrays (Adjacent Diff ±1)

ASKED 2026-02-08 | 254 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 208/228

ABOUT THIS QUESTION

Topic: More practice - Two pointers and sliding window

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-02-08 | Roles: MLE, SWE

Original: https://www.1point3acres.com/interview/problems/company/google/good-subarrays-diff-one

QUESTION

MLE L5 coding round: sum over all contiguous subarrays whose elements form a strictly monotonic ±1 step sequence (every adjacent pair differs by exactly +1, OR every adjacent pair differs by exactly -1). Required time complexity is O(N).

Input: integer array arr of length N.
A subarray is good if its consecutive differences are either all +1 (monotone increasing by 1) or all -1 (monotone decreasing by 1).
Singleton subarrays are always good.
Output: the sum of element-sums across all good subarrays (each good subarray contributes the sum of its elements once).
Required: O(N) time.

EXAMPLES

```
arr = [3, 5, 6, 7, 6]
```

Good subarrays

```
  singletons:           [3], [5], [6], [7], [6]
  monotone +1 runs:     [5, 6], [6, 7], [5, 6, 7]
  monotone -1 runs:     [7, 6]
Not good: [3, 5] (diff 2), [6, 7, 6] (mixes +1 then
    -1).
```

APPROACH

Practice the "contribution-per-index" pattern: the sum-of-subarray-minimums and sum-of-subsequence-widths problems both use the same technique.
Practice splitting an array into maximal monotone-step runs, then computing per-run contributions in O(1) amortized using prefix sums of element values.
Sanity-check with arr = [1, 2, 3] and arr = [3, 2, 1] by hand — both should sum to 20.

HOW TO THINK ABOUT IT

A good subarray is a stretch where every step is +1 or every step is -1, so every good subarray ending at index i lies inside the current run. Instead of listing subarrays, keep for each direction the total of all suffix sums inside the run: extending a run of length L by a new element x adds (L + 1) * x to that total. Both directions can be updated in O(1), and the singleton is counted once.

Basis: Written for this question (a run-length DP in the style of LeetCode 2262, Total Appeal of A String).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    total = 0
    for i in range(len(a)):
        for j in range(i, len(a)):
            part = a[i:j + 1]
            if is_good(part):
                total += sum(part)
    return total
def is_good(part):
    pairs = zip(part, part[1:])
    steps = {y - x for x, y in pairs}
    return steps <= {1} or steps <= {-1}
```

TIME	SPACE
O(n^2) or O(n^3)	O(1)

WHY IT IS SLOW

O(n^2) subarrays (O(n^3) with the sums). Run totals per direction update in O(1) per element, so one pass is O(n).

OPTIMAL SOLUTION · Two running run totals

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is the singleton subtracted?

A one-element subarray [x] is both an increasing and a decreasing run of length 1, so it appears in both running totals. Subtracting x once at each index keeps it counted exactly once. A subarray of length 2 or more can only be in one total because its step is +1 or -1, not both.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
# MAIN QUESTION: Sum of Good Subarrays (Adjacent Diff ±1)
#
# MLE L5 coding round: sum over all contiguous subarrays
# whose elements form a strictly monotonic ±1 step
# sequence (every adjacent pair differs by exactly +1, OR
# every adjacent pair differs by exactly -1). Required
# time complexity is O(N). Input: integer array arr of
# length N. A subarray is good if its consecutive
# differences are either all +1 (monotone increasing by 1)
# or all -1 (monotone decreasing by 1). Singleton
# subarrays are always good. Output: the sum of element-
# sums across all good subarrays (each good subarray
# contributes the sum of its elements once). Required:
# O(N) time. Examples: arr = [3, 5, 6, 7, 6] Good
# subarrays: singletons: [3], [5], [6], [7], [6] monotone
# +1 runs: [5, 6], [6, 7], [5, 6, 7] monotone -1 runs: [7,
# 6] Not good: [3, 5] (diff 2), [6, 7, 6] (mixes +1 then
# -1).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Why is the singleton subtracted?
#
# INTERVIEW NOTES
# A good subarray is a stretch where every step is +1 or
# every step is -1, so every good subarray ending at index
# i lies inside the current run. Instead of listing
# subarrays, keep for each direction the total of all
# suffix sums inside the run: extending a run of length L
# by a new element x adds (L + 1) * x to that total. Both
# directions can be updated in O(1), and the singleton is
# counted once.
#
# BRUTE FORCE (say it first): Enumerate every subarray
# Check every subarray's differences and add up its
# elements. Correct and the oracle; quadratic or worse,
# but the problem asks for O(n). Time O(n^2) or O(n^3).
# Space O(1).


# BRUTE FORCE (checker)
def good_subarray_sum_brute(a):
    # check every subarray: the oracle for the demo
    total = 0
    for i in range(len(a)):
        for j in range(i, len(a)):
```

```python
            steps = {a[k + 1] - a[k] for k in range(i, j)}
            if steps <= {1} or steps <= {-1}:
                total += sum(a[i:j + 1])
    return total


# BASE QUESTION (optimal): Two running run totals
# Idea: For an increasing run of length L ending at i let
# S be the sum of all suffix sums of the run. When the
# next element x extends it, S becomes S + (L + 1) * x and
# L becomes L + 1; a break restarts at S = x, L = 1.
def good_subarray_sum(a):
    # a good subarray has all steps +1 or all steps -1.
    # For each index keep, per direction, the sum of the
    # sums of all good subarrays that END here.
    total = 0
    up_sum = up_len = down_sum = down_len = 0
    for i, x in enumerate(a):
        step = x - a[i - 1] if i else None
        if step == 1:
            # extend the +1 run: every old subarray grows
            # by
            # x, and [x] alone is new: (len + 1) * x in
            # all
            up_sum += (up_len + 1) * x
            up_len += 1
        else:
            up_sum, up_len = x, 1
```

```python
        if step == -1:
            down_sum += (down_len + 1) * x
            down_len += 1
        else:
            down_sum, down_len = x, 1
        # [x] is in both totals: count it once
        total += up_sum + down_sum - x
    return total


if __name__ == "__main__":
    arr = [3, 5, 6, 7, 6]
    # singletons 27 + [5,6] 11 + [6,7] 13 + [5,6,7] 18 +
    # [7,6] 13
    assert good_subarray_sum(arr) == 82
    assert good_subarray_sum([]) == 0
    assert good_subarray_sum([4]) == 4
```

```python
    # the O(n) answer matches the brute force everywhere
    import itertools
    for n in range(7):
        every = itertools.product([1, 2, 3, 4], repeat=n)
        for a in every:
            assert good_subarray_sum(list(a)) == (
                good_subarray_sum_brute(list(a)))
```

NOTES FROM CANDIDATES

Walk the array maintaining two run lengths: upRun (current trailing run of +1 steps) and downRun (current trailing run of -1 steps). At index i, only one of them is nonzero (the other resets to 1).
Each index i ends upRun + downRun - 1 good subarrays (subtracting the double-counted singleton). For each ending position, contribute the running prefix-sum tail of the active run.
Multiple candidates report getting stuck — verbalize the "contribution per index" framing out loud before coding.
Easy to mis-handle the reset boundary; double-check with a 2-element example before writing the loop.

PREPARATION

Practice the "contribution-per-index" pattern: the sum-of-subarray-minimums and sum-of-subsequence-widths problems both use the same technique.
Practice splitting an array into maximal monotone-step runs, then computing per-run contributions in O(1) amortized using prefix sums of element values.
Sanity-check with arr = [1, 2, 3] and arr = [3, 2, 1] by hand — both should sum to 20.

STUDY PATH: question 120 of 167 (more practice)

Priority score 2.0 | Two pointers and sliding window | asked 170 days ago | frequency Single report

HARD | 1Point3Acres | onsite-coding | 3 of 4

Necklace Cut into Two Halves with Equal D/R Counts

ASKED 2026-05-03 | 170 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 188/228

ABOUT THIS QUESTION

Topic: More practice - Two pointers and sliding window

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-05-03 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/necklace-cut-equal-counts

QUESTION

EXAMPLES

```
s = "DRDR"    -> cut between index 1 and 2: "DR" +
    "DR".
s = "DDRR"    -> 1-cut not balanced ("DD" vs "RR");
    rotate, two cuts at offsets 1 and 3:
              pieces "DR" and "DR".
```

APPROACH

Practice the running-balance technique on +1/-1 arrays (the contiguous-array and subarray-sums-divisible-by-k families). Drill sliding-window-on-circular-array unrolling (duplicate the string s+s and use a window of length n).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the string is circular, whether the two pieces must be contiguous arcs, whether equal counts of each letter or equal strings are needed, and whether one cut or two are allowed.

BRUTE FORCE TO SAY OUT LOUD

Try every pair of cut positions and count letters. O(n^3) or O(n^2) with prefix counts.

OPTIMAL IDEA

Each arc must hold half of the D's and half of the R's, so the total counts of both must be even and each arc has length n / 2. Slide a window of length n / 2 around s + s tracking its D count; the window starting at i and the one starting at i + n / 2 are complementary, and counts change by at most one per step, so some start hits exactly half. O(n).

DRY-RUN THESE CASES

DRDR, DDRR (two cuts needed), an odd D count (impossible), a string of one letter.

LIKELY FOLLOW-UPS

Equal strings instead of equal counts, or three pieces.

PITFALLS

INTERVIEW-DAY PLAN

Assuming a linear cut is always enough. Ignoring parity.

HOW TO THINK ABOUT IT

Two arcs of a circle with the same letter counts must each have exactly half the D's and half the R's, so the total counts must be even and each arc has length n / 2. Slide a window of length n / 2 around the circle: the window and its complement have D counts adding to the total, and the count changes by at most one per step, so it must hit half somewhere.

Basis: Circular sliding window / prefix balance (cf. LeetCode 525, Contiguous Array). Assumption: cut the circular necklace at two points into two arcs, each with the same number of D and R.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s):
    n = len(s)
    for i, j in combinations(range(n), 2):
        arc1 = s[i:j]
        arc2 = s[j:] + s[:i]
        d1, d2 = arc1.count("D"), arc2.count("D")
        r1, r2 = arc1.count("R"), arc2.count("R")
        if d1 == d2 and r1 == r2:
            return True
    return False
```

TIME	SPACE
O(n^3)	O(n)

WHY IT IS SLOW

O(n^2) pairs and each recount costs O(n), so O(n^3). Because the arcs must both have length n / 2, one circular window of that length slid around the string finds it in O(n).

OPTIMAL SOLUTION · Fixed-length circular window with the discrete intermediate value argument

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 When is a single cut enough?

A single linear cut works when some prefix has exactly half the D's and half the R's, i.e. a prefix of length n/2 whose D count is d_total / 2. Otherwise rotate: the circular window above finds the start, which is equivalent to two cuts on the circle.

FOLLOW-UP 2 What if the two pieces must be equal strings, not just equal counts?

That is a different question: the necklace must be a square rotation, i.e. some rotation of s is t + t. Check whether s appears inside (s + s)[1:-1] at index n/2, or compare s[i:i + n/2] with the complementary arc using a rolling hash or KMP over s + s. It is a string matching problem, not a counting one.

FOLLOW-UP 3 The two pieces must be equal strings, not just equal counts.

Then the necklace must be a square in some rotation: some rotation of s equals t + t. Test with s in (s + s)[1:-1] restricted to an even-length split, or compare s[i:i + n/2] with s[i + n/2:i + n] using a rolling hash or KMP.

FOLLOW-UP 4 The two pieces must be equal strings, not just have equal counts.

A rotation of the necklace must be of the form t + t. Try every rotation and compare its two halves (O(n^2), or O(n) by searching s in (s + s)[1:-1] and checking the match at an offset of n / 2).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Necklace Cut into Two Halves with Equal
# D/R Counts
#
# Examples: s = "DRDR" -> cut between index 1 and 2: "DR"
+
# "DR". s = "DDRR" -> 1-cut not balanced ("DD" vs "RR");
# rotate, two cuts at offsets 1 and 3: pieces "DR" and
# "DR".
#
# FOLLOW-UPS (answered below, in this order):
# 1. When is a single cut enough?
# 2. What if the two pieces must be equal strings, not
# just equal counts?
# 3. The two pieces must be equal strings, not just equal
# counts.
# 4. The two pieces must be equal strings, not just have
# equal counts.
#
# INTERVIEW NOTES
# Two arcs of a circle with the same letter counts must
# each have exactly half the D's and half the R's, so the
# total counts must be even and each arc has length n / 2.
# Slide a window of length n / 2 around the circle: the
# window and its complement have D counts adding to the
# total, and the count changes by at most one per step, so
# it must hit half somewhere.
#
# BRUTE FORCE (say it first): Try every pair of cut points
# Choose two cut positions, form the two arcs, and compare
# their letter counts. Correct and the oracle; it checks
# every pair of cuts and recounts letters each time. Time
# O(n^3). Space O(n).
```

```
# BRUTE FORCE (say it first): Try every pair of cut points
# | time O(n^3)
def necklace_brute(s):
    n = len(s)
    for i in range(n):
        for j in range(i + 1, n):
            a, b = s[i:j], s[j:] + s[:i]
            if (
                a
                and b
                and a.count("D") == b.count("D")
                and a.count("R") == b.count("R")
            ):
                return True
    return False


# BASE QUESTION (optimal): Fixed-length circular window
# with the discrete intermediate value argument
# Idea: If each arc has half of the D's and half of the
# R's, each arc has length n/2, and the total D and R
# counts must both be even. Slide a window of length n/2
# around the circle (use s + s), tracking its D count.
def necklace_two_cuts(s):
    n = len(s)
    d_total, r_total = s.count("D"), s.count("R")
    # each piece needs half of both letters
    if n % 2 or d_total % 2 or r_total % 2:
        return None
    half, target = n // 2, d_total // 2
    doubled = s + s
    # slide a window of length n/2 around the circle
    window = doubled[:half].count("D")
    for i in range(n):
        if window == target:
            # cuts at i and i + n/2 give two good halves
            first = doubled[i:i + half]
            second = doubled[i + half:i + n]
            return i, (i + half) % n, first, second
        # move the window one step: add the new, drop the
        # old
        window += (doubled[i + half] == "D")
        window -= (doubled[i] == "D")
    return None


# SINGLE CUT case
def single_cut(s):
    # one cut on the line: a prefix with half of
    # everything
    half = len(s) // 2
    if len(s) % 2:
        return None
    d_prefix = s[:half].count("D")
    return half if d_prefix * 2 == s.count("D") else None
```

```
# FOLLOW-UP 2: What if the two pieces must be equal
# strings, not just equal counts?
# Answer: That is a different question: the necklace must
# be a square rotation, i.e. some rotation of s is t + t.
# FOLLOW-UP 3: The two pieces must be equal strings, not
# just equal counts.
# Answer: Then the necklace must be a square in some
# rotation: some rotation of s equals t + t. Test with s
# in (s + s)[1:-1] restricted to an even-length split, or
# compare s[i:i + n/2] with s[i + n/2:i + n] using a
# rolling hash or KMP.
# FOLLOW-UP 4: The two pieces must be equal strings, not
# just have equal counts.
# Answer: A rotation of the necklace must be of the form t
# + t. Try every rotation and compare its two halves
```

```python
# (O(n^2), or O(n) by searching s in (s + s)[1:-1] and
# checking the match at an offset of n / 2).
def equal_string_halves(s):
    # follow-up: the two pieces must be the same string
    n = len(s)
    if n % 2:
        return None
    half, doubled = n // 2, s + s
    for i in range(n):
        if doubled[i:i + half] == doubled[i + half:i + n]:
            return i, (i + half) % n
    return None


if __name__ == "__main__":
    # DRDR: one cut between the halves works
    assert single_cut("DRDR") == 2
    cuts = necklace_two_cuts("DRDR")
    assert cuts is not None and cuts[2].count("D") == 1
    # DDRR needs two cuts on the circle: DR + DR
    assert single_cut("DDRR") is None
    i, j, a, b = necklace_two_cuts("DDRR")
    assert sorted(a) == sorted(b) == ["D", "R"]
    # odd totals have no answer
    assert necklace_two_cuts("DDR") is None
    assert necklace_two_cuts("DRDRD" + "R") is None
    # equal strings: DRDR is DR + DR, DDRR is not
    assert equal_string_halves("DRDR") is not None
    assert equal_string_halves("DDRR") is None
```

NOTES FROM CANDIDATES

Why 2 cuts always suffice: think of a running balance cnt(D) - cnt(R); over the full string it returns to 0. Any prefix splits the necklace into two arcs whose imbalances sum to 0; if the imbalance at some prefix is already 0, one cut suffices, otherwise a sliding window of length n/2 proves a 2-cut works.
Sliding-window approach: fix one cut at index i, look for j such that the window [i, j) has half the Ds and half the Rs. As i advances, j moves monotonically.
Don't get distracted by the necklace framing; once unwrapped onto a line of length 2n and constrained to half-length windows, the problem is standard.
Verbalize the 2-cut bound out loud before coding — interviewers ask why.

PREPARATION

Practice the running-balance technique on +1/-1 arrays (the contiguous-array and subarray-sums-divisible-by-k families).
Drill sliding-window-on-circular-array unrolling (duplicate the string s+s and use a window of length n).

STUDY PATH: question 121 of 167 (more practice)

Priority score 1.72 | Two pointers and sliding window | asked 316 days ago

HARD | IN APP LC 1044 | PracHub | Technical Screen | 4 of 4

Find longest duplicated substring

POSTED 2025-12-08 | 316 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 30.5 | 30 DAYS 0 | PRIORITY 122/228

ABOUT THIS QUESTION

Topic: More practice - Two pointers and sliding window

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1044 Longest Duplicate Substring

LeetCode match: LC 1044 Longest Duplicate Substring

Posted on PracHub: 2025-12-08 (updated 2026-07-09)

Original: https://prachub.com/coding-questions/find-longest-duplicated-substring

QUESTION

You are given a string **s** consisting of lowercase English letters. A **substring** of **s** is any contiguous sequence of characters, i.e., `s[i..j)` for `0 <= i < j <= len(s)`. Design an algorithm to find **any one** of the longest substrings that appears **at least twice** in **s**. The two occurrences may overlap. If no substring appears at least twice, return the empty string.

Requirements

- Input: a string **s** of length **n** (e.g., `1 <= n <= 2 × 10^5`).
- Output: a string representing one of the longest duplicated substrings.
- Aim for an algorithm substantially better than the naive `O(n^2)` substring comparison approach.

Examples

- `s = "banana"`
- Possible duplicated substrings: "a", "an", "ana" - Longest length is 3 ("ana"), so acceptable outputs include `"ana"`.
- `s = "abcd"`
- No duplicated substring, so output should be `""` (empty string). Describe your algorithm, its time and space complexity, and how it scales for large **n**.

APPROACH

Binary search on the substring length; check each length with a Rabin-Karp rolling hash plus a set (verify on collision). Alternative: suffix array + LCP. O(n log n).

HOW TO THINK ABOUT IT

If a substring of length L repeats, then a substring of every shorter length repeats too, so 'is there a duplicate of length L' is monotone and the answer length can be binary searched. Each check slides a window of length L over the string with a rolling hash and looks for a repeated hash, verifying the match to guard against collisions.

Basis: LeetCode 1044, Longest Duplicate Substring (binary search on the length with a rolling hash).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(s):
    n = len(s)
    for L in range(n - 1, 0, -1):
        seen = set()
        for i in range(n - L + 1):
            sub = s[i:i + L]
            if sub in seen:
                return sub
            seen.add(sub)
    return ""
```

TIME	SPACE
O(n^3)	O(n^2)

WHY IT IS SLOW

Each length builds up to n substrings of up to n characters: O(n^3) time and O(n^2) space. If a duplicate of length L exists then so does one of every shorter length, so binary search on L with a rolling hash is O(n log n).

OPTIMAL SOLUTION · Binary search on the length with a rolling hash (Rabin-Karp)

TIME	SPACE
O(n log n) expected	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Avoid hash collisions entirely.

Use a suffix array with the LCP array: the longest duplicate is the largest LCP between neighbours in suffix order. Naive construction (sorting suffixes) is O(n^2 log n); real suffix array algorithms are O(n log n) or O(n).

FOLLOW-UP 2 The substring must appear at least k times, or must not overlap.

At least k times: count equal hashes in the check (need k of them). Non-overlapping: when two windows match, require their start positions to differ by at least L.

FOLLOW-UP 3 The string is huge and does not fit in memory.

Hash windows on disk in passes, or use a suffix automaton built online; for approximate answers use minhash sketches. State that exact answers need memory proportional to the number of distinct windows. **Cost:** O(n log n) time and O(n) memory for the hashes; on disk the hashing passes multiply the I/O by about log n.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find longest duplicated substring
#
# You are given a string s consisting of lowercase English
# letters. A **substring** of s is any contiguous sequence
# of characters, i.e., s[i..j) for 0 <= i < j <= len(s).
# Design an algorithm to find **any one** of the longest
# substrings that appears **at least twice** in s. The two
# occurrences may overlap. If no substring appears at
# least twice, return the empty string. ### Requirements -
# Input: a string s of length n (e.g., 1 <= n <= 2 × 10^5)
.
# - Output: a string representing one of the longest
# duplicated substrings. - Aim for an algorithm
# substantially better than the naive O(n^2) substring
# comparison approach. ### Examples - s = "banana" -
# Possible duplicated substrings: "a", "an", "ana" -
# Longest length is 3 ("ana"), so acceptable outputs
# include "ana". - s = "abcd" - No duplicated substring,
# so output should be "" (empty string).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Avoid hash collisions entirely.
# 2. The substring must appear at least k times, or must
# not overlap.
# 3. The string is huge and does not fit in memory.
#
# INTERVIEW NOTES
# If a substring of length L repeats, then a substring of
# every shorter length repeats too, so 'is there a
# duplicate of length L' is monotone and the answer length
# can be binary searched. Each check slides a window of
# length L over the string with a rolling hash and looks
# for a repeated hash, verifying the match to guard
# against collisions.
#
# BRUTE FORCE (say it first): Check every length from
# longest down with a set of substrings
# For each length from n - 1 down, put all substrings of
# that length in a set until one repeats. Correct and the
# oracle; every check builds up to n substrings of length
# up to n. Time O(n^3). Space O(n^2).


# BRUTE FORCE (say it first): Check every length from
# longest down with a set of substrings | time O(n^3)
def longest_dup_brute(s):
```

```
    for L in range(len(s) - 1, 0, -1):
        seen = set()
        for i in range(len(s) - L + 1):
            sub = s[i : i + L]
            if sub in seen:
                return sub
            seen.add(sub)
    return ""


# HELPERS for the base question
MOD = (1 << 61) - 1


BASE = 131


def find_length(s, length):
    # a repeated window of this length? (rolling hash)
    codes = [ord(c) for c in s]
    power = pow(BASE, length, MOD)
    h = 0
    for i in range(length):
        h = (h * BASE + codes[i]) % MOD
    seen = {h: [0]}
    for i in range(1, len(s) - length + 1):
        # slide the window: drop the left, add the right
        h = (h * BASE - codes[i - 1] * power
             + codes[i + length - 1]) % MOD
        # a hash match may be a collision: compare the
        # text
        for j in seen.get(h, ()):
            if s[j:j + length] == s[i:i + length]:
                return s[i:i + length]
        seen.setdefault(h, []).append(i)
    return None


# BASE QUESTION (optimal): Binary search on the length
# with a rolling hash (Rabin-Karp)
# Idea: For a fixed length L compute the polynomial hash
# of every window in O(1) each by sliding it, store hashes
# with their start positions, and when a hash repeats
# compare the actual substrings (to rule out a collision).
def longest_dup(s):
```

```
    # binary search the length: if length L repeats, so do
    # all shorter lengths
    lo, hi, best = 1, len(s) - 1, ""
    while lo <= hi:
        mid = (lo + hi) // 2
        found = find_length(s, mid)
        if found:
            best, lo = found, mid + 1
        else:
            hi = mid - 1
    return best


# FOLLOW-UP 1: Avoid hash collisions entirely.
# Answer: Use a suffix array with the LCP array: the
# longest duplicate is the largest LCP between neighbours
# in suffix order. Naive construction (sorting suffixes)
# is O(n^2 log n); real suffix array algorithms are O(n
# log n) or O(n).
def longest_dup_suffix(s):
    # no hashing: sort the suffixes, compare neighbours
    # (the
    # simple O(n^2 log n) version of a suffix array)
    suffixes = sorted(s[i:] for i in range(len(s)))
    best = ""
    for a, b in zip(suffixes, suffixes[1:]):
        k = 0
        while k < min(len(a), len(b)) and a[k] == b[k]:
            k += 1
        if k > len(best):
            best = a[:k]
    return best
```

```
# FOLLOW-UP 2: The substring must appear at least k times,
# or must not overlap.
# Answer: At least k times: count equal hashes in the
# check (need k of them). Non-overlapping: when two
# windows match, require their start positions to differ
# by at least L.
def longest_repeated(s, times=2, non_overlapping=False):
    # at least `times` occurrences; optionally not
    # overlapping
    best = ""
    n = len(s)
```

```
    for length in range(1, n):
        spots = {}
        for i in range(n - length + 1):
            piece = s[i:i + length]
            spots.setdefault(piece, []).append(i)
        ok = None
        for text, starts in spots.items():
            if non_overlapping:
                # greedy: count starts at least `length`
                # apart
                taken, last = 0, -length
                for p in starts:
                    if p - last >= length:
                        taken, last = taken + 1, p
            else:
                taken = len(starts)
            if taken >= times:
                ok = text
                break
        if ok is None:
            break
        best = ok
    return best


if __name__ == "__main__":
    assert longest_dup("banana") in ("ana",)
    assert longest_dup("abcd") == ""
    assert len(longest_dup("aaaaa")) == 4
    assert len(longest_dup_suffix("banana")) == 3
    assert longest_dup_suffix("abcd") == ""
    for s in ("abracadabra", "mississippi", "abab"):
        fast = len(longest_dup(s))
        assert fast == len(longest_dup_suffix(s))
    # three occurrences needed
    assert longest_repeated("banana", 3) == "a"
    # overlapping copies are not allowed: "aa" cannot fit
    # in aaaa
    assert len(longest_repeated("aaaa", 2, True)) == 2
    assert len(longest_repeated("aaaa", 2)) == 3
```

FUNCTION AND TEST CASES

Signature

```
def solution(s) -> str:
```

Sample call

```
solution('banana')
```

Test cases

Input	Expected
`('banana',)`	`'ana'`
`('abcd',)`	`''`